GeoMinerAI

AI-Augmented Geological Exploration Workspace

Technical Documentation & Architecture Guide

Version 2.0 | RAG . Hotspot . ML Prospectivity . Geological Cross-Section . Geoprocessing

github.com/sehmilo/geominerAI_GeoGPT

What is GeoMinerAI?

Agent, Tool, or Both?

GeoMinerAI is best described as an NLP-driven domain-specific AI co-pilot for mineral exploration geology. It is neither a pure tool nor a full autonomous agent. It occupies a hybrid position:

AS A TOOL	AS AN AGENT (partially)	NOT a full agent
• Streamlit web app with graphical UI	• Interprets NL commands, selects pipelines autonomously	• No ReAct / planning loop
• Takes discrete inputs, returns discrete outputs	• Orchestrates sub-tasks without step-by-step guidance	• No self-correction or autonomous goals
• Does not act without user prompting	• Maintains state: layers, chat history, drawn features	• Intent routing is deterministic keyword/regex
	• Routes to specialised tools based on detected intent	• LLM called for generation only, not orchestration

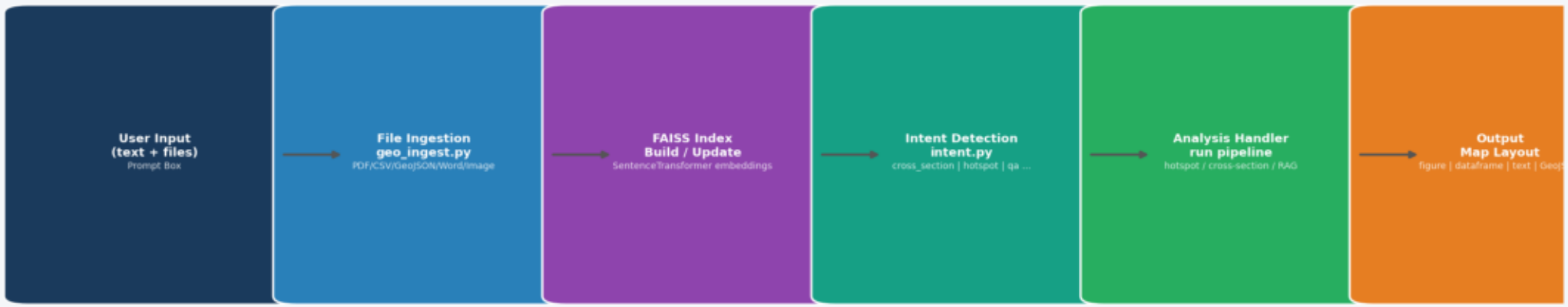
Formal label: NLP-routed analytical workspace with agentic UI behaviour and domain-specific geological intelligence.

System Architecture

Core Modules

Module	Status	Purpose
`geo_ingest.py`	**NEW**	Multi-format ingestion: PDF, Word, CSV, GeoJSON, KML, image, raster
`intent.py`	**NEW**	NLP dispatcher: keyword+regex -> (intent, metadata)
`cross_section.py`	**NEW**	LLM cross-section prompt + matplotlib schematic figure
`geoprocessing.py`	**NEW**	Buffer (Shapely), clip, centroid, haversine distance
`rag.py`		PDF chunking, FAISS index build, semantic retrieval
`llm_hf.py`		HuggingFace InferenceClient: text_generation + chat_completion
`snree_hotspots.py`		Getis-Ord Gi*, Local Moran's I, grid-binning spatial stats
`snree_ml.py`		Logistic Regression + Random Forest + within-hole depth trend
`snree_lab.py`		Sn-REE prospectivity: XRF summary, FracProxy, screening
`chemistry.py`		Element-to-oxide conversion, column standardisation

Data Flow



1. User Input

Text command + file upload via Prompt Box

2. Ingestion

geo_ingest routes by extension; creates layer dict

3. Index Build

SentenceTransformer encodes chunks; FAISS rebuilt

4. Intent

detect_intent() -> (intent, metadata); no LLM call

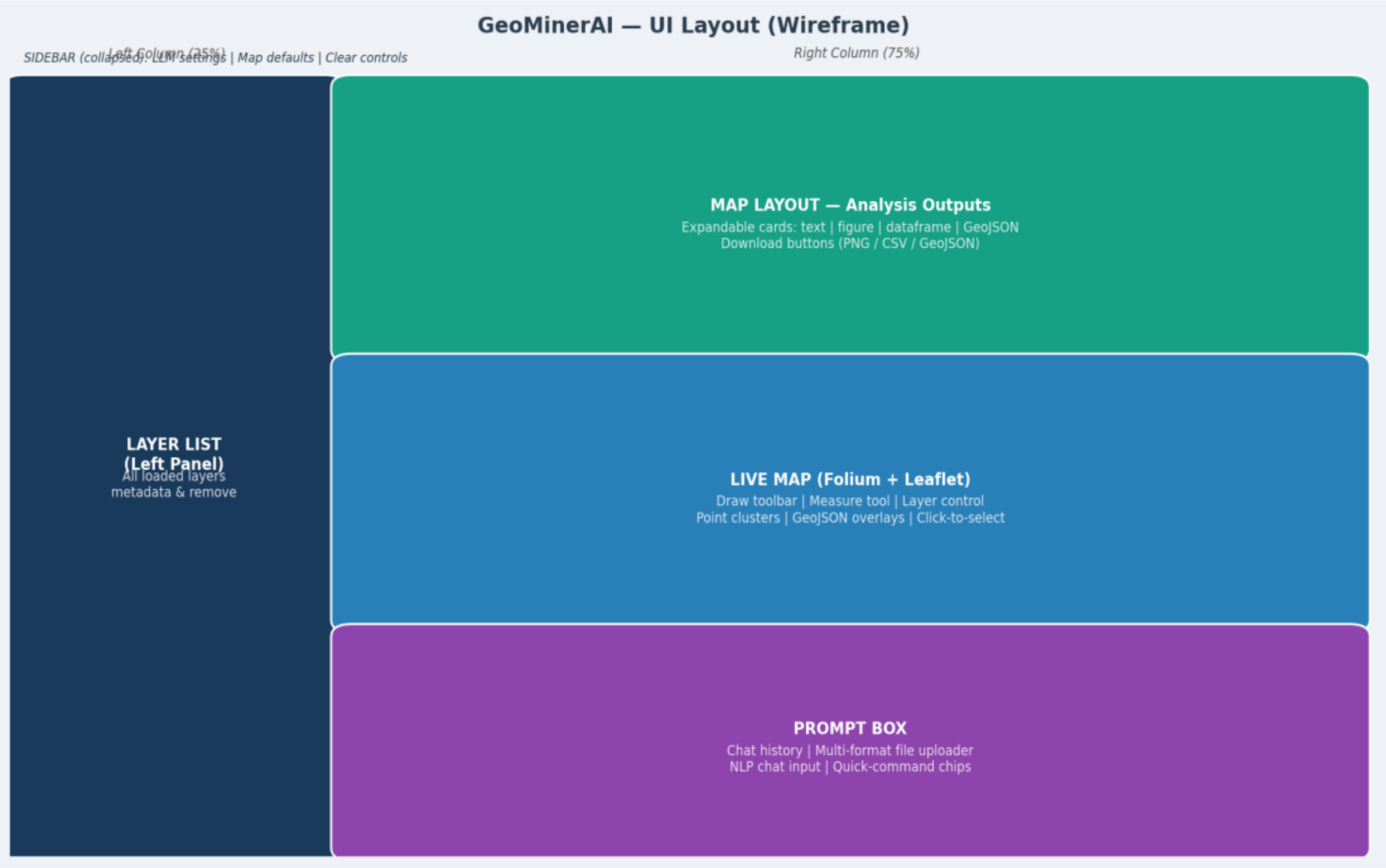
5. Handler

Pipeline executes; _push_output() stores result

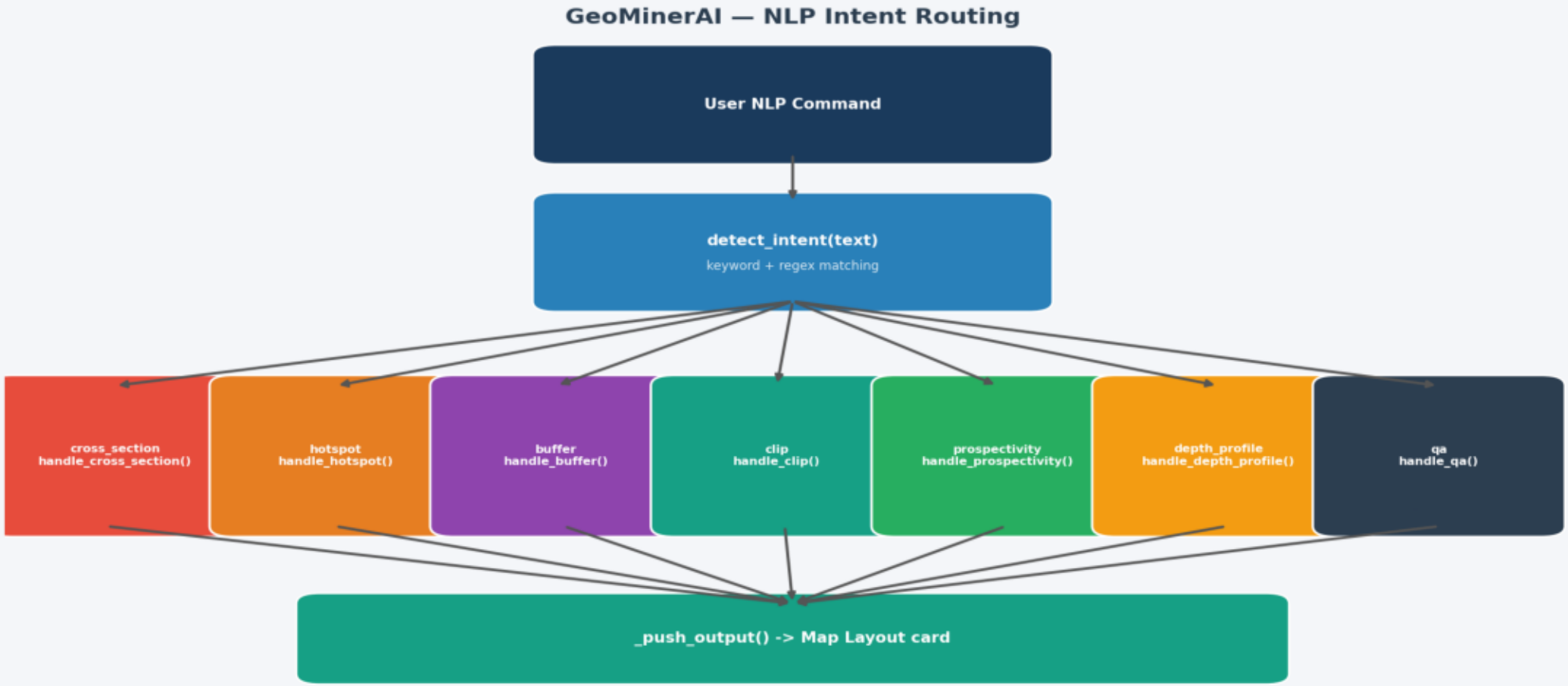
6. Render

Streamlit reruns; Map Layout shows new card

UI Layout


GeoMinerAI — UI Layout (Wireframe)
SIDEBAR (collapsed): LLM settings | Map defaults | Clear controls
Left Column (25%)
Right Column (75%)
LAYER LIST
(Left Panel)
All loaded layers
metadata & remove
MAP LAYOUT — Analysis Outputs
Expandable cards: text | figure | dataframe | GeoJSON
Download buttons (PNG / CSV / GeoJSON)
LIVE MAP (Folium + Leaflet)
Draw toolbar | Measure tool | Layer control
Point clusters | GeoJSON overlays | Click-to-select
PROMPT BOX
Chat history | Multi-format file uploader
NLP chat input | Quick-command chips

NLP Intent Routing



cross_section

'cross section', 'section line'

hotspot

'hotspot', 'Gi*', 'Moran'

buffer

'buffer', 'zone around'

clip

'clip', 'extract within'

prospectivity

'prospectivity', 'Sn-REE'

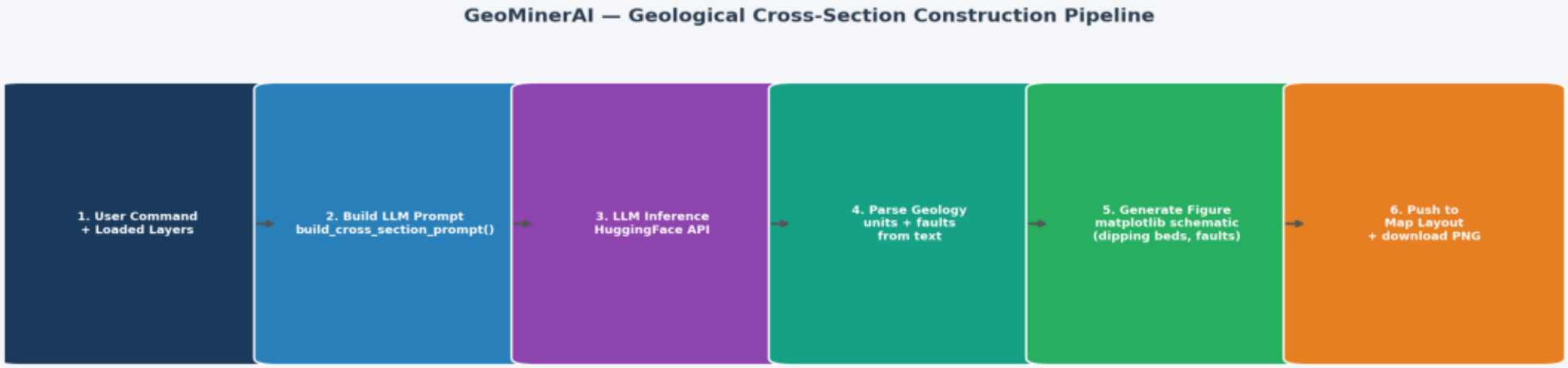
depth_profile

'depth profile', 'downhole'

qa

(fallback)

Geological Cross-Section Engine



- Establish section orientation: true vs apparent dip; apparent-dip correction formula
- Construct topographic profile: extract or estimate elevations; state vertical exaggeration
- Project surface contacts: section-line intersections with lithological contacts
- Project units to depth: dip-based projection; data-constrained vs inferred geometry clearly labelled
- Interpret structures: folds, fault types (normal/reverse/thrust/strike-slip), displacement magnitude
- Handle uncertainty: plausible model + at least two alternative structural interpretations
- Validate internal consistency: impossible stacking, thickness errors, fault displacement errors
- Mineral exploration relevance: structural traps, intersection zones, optimal drill targets

Advantages

Unified workspace

All workflows on one page. No tab switching, no context loss.

Natural language

Domain commands understood instantly: 'run hotspot', 'draw cross section'.

Multi-format KB

PDF, Word, CSV, GeoJSON, KML, images, rasters - all auto-indexed.

Cross-section engine

No equivalent open-source tool: LLM interpretation + matplotlib schematic.

Under the hood

Users see final outputs. Gi*, FAISS, ML - all invisible.

Spatial editing

Draw polygons on map -> use in buffer/clip commands immediately.

Transparent outputs

Every result: download PNG / CSV / GeoJSON.

Modular design

New analysis: add handler + keyword in intent.py. No restructuring.

Limitations & Challenges

Streamlit Performance

- Full page rerun on every interaction (0.5-8s penalty)
- Blocking execution: analysis freezes entire UI thread
- Session state memory grows unbounded
- No multi-user isolation for shared model memory

Map Rendering (Folium)

- Entire Folium map rebuilt on every Streamlit rerun
- 1000+ markers cause browser slowdown; 5000+ = unresponsive
- iframe round-trip latency: 200-500ms per map interaction
- Drawing state partially lost between reruns

LLM Constraints

- HuggingFace free-tier rate limits and 503 errors
- No token streaming - spinner waits 5-30s for response
- Open-weight model quality varies for geological tasks
- Context window limits may miss KB content

Data & Scale

- No persistent storage: data lost on page refresh
- FAISS rebuilt in-memory on every layer addition
- No native shapefile support without GDAL binary
- Cross-section parser may fail on complex input

Proposed Fixes & Mitigations

Immediate fixes (within Streamlit):

- @st.cache_resource - wrap SentenceTransformer load; saves 2-5s per rerun
- @st.cache_data - cache hotspot + ML results; repeated analysis returns instantly
- st.fragment (Streamlit >=1.37) - isolate map reruns from the rest of the page
- MarkerCluster plugin - replace individual markers for datasets >500 points
- pydeck ScatterplotLayer - WebGL rendering of 100k+ points at interactive framerates
- Background thread for LLM - non-blocking inference with live progress indicator

Architectural upgrades (beyond Streamlit):

- FastAPI + React/Next.js - partial component updates; no full-page reruns
- WebSocket LLM streaming - token-by-token display; eliminates spinner wait
- PostgreSQL + Redis - persistent layers and session state across restarts
- Celery task queue - offload LLM + ML to background workers
- PostGIS + Vector Tiles - serve millions of points via MVT for smooth map rendering

Future Directions & Roadmap

Near-term (3-6 months)

- st.fragment map isolation
- pydeck WebGL point rendering
- Persistent SQLite/PostgreSQL storage
- Streaming LLM via st.write_stream
- Shapefile/GeoPackage support
- DXF export for CAD
- Drillhole 3D viewer

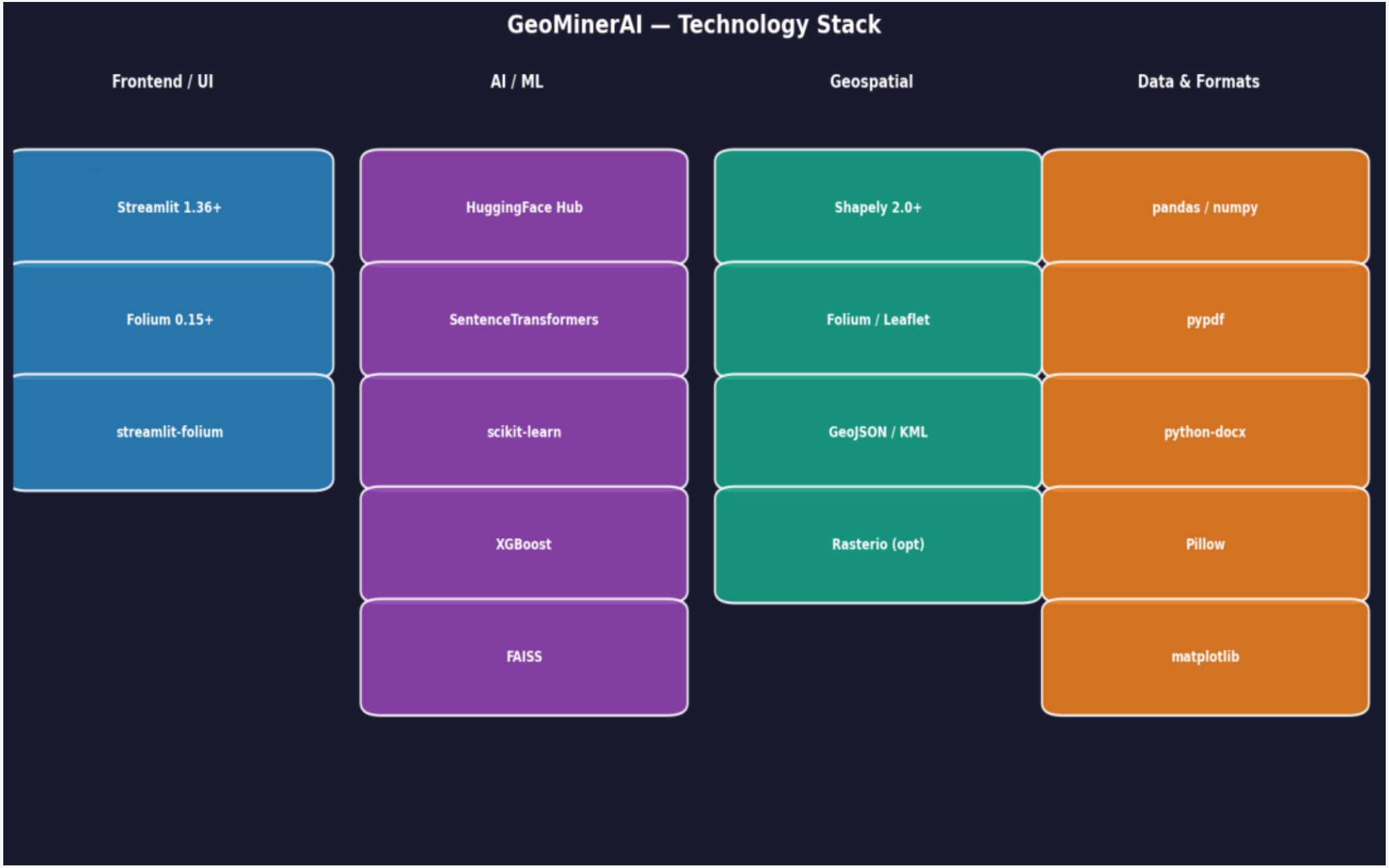
Medium-term (6-18 months)

- FastAPI + React frontend
- Multi-user collaboration + real-time sync
- 3D modelling (CesiumJS / Three.js)
- Geological database integration
- ReAct-style LLM agent
- Domain-fine-tuned geological LLM

Long-term Vision

- Mobile GPS field companion app
- LiDAR / drone point-cloud ingestion
- Geostatistical resource estimation (kriging)
- Export to Leapfrog / Seequent / Micromine
- One-click exploration report generation

Technology Stack



Quick-Start Guide

Installation:

```
$ git clone https://github.com/sehmilo/geominerAI_GeoGPT.git

$ cd geominerAI_GeoGPT

$ pip install -r requirements.txt

$ echo 'HF_TOKEN = "hf_xxxx"' > .streamlit/secrets.toml

$ streamlit run app.py
```

Workflow 1 - Hotspot Analysis:

- Upload CSV: hole_id, lat, lon, SnO2
- Click Hotspot chip or type: run hotspot analysis on SnO2
- Cluster map appears on Live Map; table in Map Layout
- Download CSV from the output card

Workflow 2 - Geological Cross-Section:

- Upload geological PDF, report, or CSV with strike/dip
- Draw section trace on Live Map; click Save drawings
- Type: draw cross section A to A prime - granite 25 NE over schist